DESIGN, NOT DEFAULTS.

How to make AI-built applications feel specific, coherent, and intentionally designed.

A specification you can give directly to an AI builder.

Five supplied articles synthesized into design rules, a design-to-build workflow, a quality gate, and ready-to-use generation and review prompts.

VERSION 1.0 • OCTOBER 2026

REFERENCE + WORKING PROMPTS

READ FIRST

The operating contract

This document is meant to be read by the AI before it designs or implements a screen. Its purpose is not to hide AI assistance; it is to prevent unexamined defaults from being shipped as design decisions.

The governing principle

Design should explain this product, for these people, doing these tasks. If a choice exists mainly because it is common in AI-generated UI, replace it with a choice justified by the workflow.

What the AI must do

□ Start with product context and user tasks, not an aesthetic adjective.

□ State visual direction and constraints before writing screen components.

□ Establish a small system of semantic tokens and reusable patterns.

□ Choose one focal point per screen and a consistent visual hierarchy.

□ Use real workflows and truthful content rather than ornamental proof.

□ Design populated, empty, loading, failure, permission, and success states.

□ Check real renders at more than one viewport; fix issues before declaring done.

What the AI must not do

□ Do not output the first plausible "modern SaaS" layout without critique.

□ Do not add gradients, glowing blobs, glass panels, emoji icons, pill badges, fake metrics, or nested cards by default.

□ Do not create six identical feature cards to avoid deciding what matters.

□ Do not use generic marketing text, repetitive headings, or filler status dots.

□ Do not turn every section into the same container and alignment pattern.

□ Do not claim a design is verified if it was not actually rendered and inspected.

How to apply: For a new product, use Sections 1–6 and the **Master Build Prompt**. For an existing product, use the **Redesign & Audit Prompt** and the **Release Gate**. These are instructions, not merely inspiration.

01 / INTENT

Decide what the product is

The strongest way to avoid a template-like interface is to make design decisions from actual product constraints rather than from vague cues such as "futuristic," "premium," or "clean." [1–5]

Build a five-minute design brief

Decide	Question the AI must answer before designing
Product	What exactly does the app let a person do? Name concrete verbs, objects and outputs.
Audience	Who uses it, on which device, and in what environment? Novices or power users?
Primary job	What single action should feel easiest on the main screen? What is second?
Trust	What real evidence or preview will make the user believe the product works?
Personality	Pick two or three useful brand traits; turn each into visible choices.
Constraints	Platform conventions, accessibility needs, data realities, and technical limits.
References	Which 2–3 real products demonstrate a *specific* useful layout or interaction?

Specify the visual rationale

- Write a one-sentence design intent: *"A quiet, photo-first journal where images carry the hierarchy and controls stay secondary."*
- Choose a recognizable but restrained identity: typography, spacing, surface treatment, one controlled accent, and perhaps one product-specific detail.
- Separate functional screens from marketing pages. A dashboard should optimize ongoing work; a landing page should explain value and prove it.
- If essential requirements are missing, make assumptions explicit and ask for clarification when they would materially alter layout or flow.

Design decision test

For every visually noticeable element, ask: What does it communicate? Why here? Why at this size? Why this color? What happens if it is removed? If the answers are weak, simplify or redesign.

02 / SYSTEM

Build a tiny design system first

Consistency is more important than copying a fashionable component library. The example values below are **suggested implementation starting points**, not numbers prescribed by the five sources. Adapt them to the platform and user. [2, 4, 5]

Semantic tokens, not disconnected styling

Element	Recommended starting constraint
Color	Base surfaces and neutrals first; one main brand accent. Separate semantic colors for errors, warnings, success and info. Never color-cycle components.
Typography	1 type family (optional contrasting display face). Distinct display/title/section/body/label/caption roles. Limit weights.
Spacing	Use a consistent 4-point or 8-point-based scale; differentiate *within group* from *between groups* gaps.
Radius	Define small/medium/large tokens. Differentiate controls from major surfaces; do not round everything to the same pill.
Borders/shadows	Prefer spacing and subtle separators. If shadows are necessary, keep them quiet and consistent. Avoid visible neon glows.
Icons	One icon family, consistent size and stroke. Use only where comprehension improves. No emoji as functional icons.
Motion	Reserved for feedback, progression and transitions. Honor reduced-motion preferences.

Example starter token spec (adapt, do not blindly copy)

Surface: background / surface / raised. **Text:** primary / secondary / disabled. **Action:** primary / focus / hover / pressed. **State:** success / warning / error / information. **Spacing:** 4, 8, 12, 16, 24, 32, 48. **Radius:** 6, 10, 16. **Typography:** display, heading, body, label, caption. Keep each token in one central source of truth.

Hierarchy over effects

- One dominant action or content region should receive visual priority; the rest should recede.
- Use size, grouping, contrast and placement to convey rank before adding color.
- Default to readable, left-aligned body text and consistent edges; use centered composition only where the message benefits.
- Aesthetic restraint must not destroy accessibility: sufficiently strong contrast and legible text remain mandatory.

03 / PATTERN LIBRARY

Remove recognizable AI defaults

These are **diagnostic signals**, not universal bans. Each pattern can work if it is justified by the product and used intentionally. The danger is their accumulation without meaning. [1–5]

01. Multiple competing neon accents

Watch for: Several equally bright pink, cyan, lime and purple treatments compete for attention.

Do instead: Use an intentional hierarchy: neutrals, one brand accent, and narrowly assigned status colors. Make the important element, not every element, stand out.

Acceptance check: Can someone identify the primary action without scanning a rainbow?

02. Purple-to-blue gradient as an identity

Watch for: A violet gradient hero or gradient text appears simply because the product is "modern" or "AI."

Do instead: Choose color from audience, brand and use case. Keep gradients only when they carry a genuine signature or functional meaning.

Acceptance check: Can you explain why this exact gradient belongs to this product?

03. Glow, aurora and decorative glass

Watch for: Blurred blobs, radial lights and translucent panels create pseudo-depth around unrelated content.

Do instead: Use type, readable surfaces and contrast for depth. Remove visual effects that neither clarify content nor respond to interaction.

Acceptance check: Does removing the glow improve focus or readability?

04. Everything in a card

Watch for: Cards are nested two to four layers deep, or every text fragment has a rounded rectangle.

Do instead: Use cards for independent, actionable or meaningfully grouped content; use space, dividers, typography, and plain lists elsewhere.

Acceptance check: Could a wrapper be removed without loss of understanding?

05. Identical icon-plus-copy feature grids

Watch for: Three or six equally sized boxes repeat icon squares, short heading, two lines of abstraction.

Do instead: Give the critical feature more space; demonstrate a real workflow. Mix screenshot, annotated example, steps, list or comparison when appropriate.

Acceptance check: Does the section show the product, or merely claim benefits?

06. Badges above everything

Watch for: Pills such as "AI-powered" and "Built for teams" precede every heading.

Do instead: Use badges for meaningful state, plan, release, availability or category. Prefer a substantive label or no badge at all.

Acceptance check: Would removing the badge lose actionable information?

07. Emoji or decorative icons in navigation

Watch for: Emojis replace product iconography; sparkle icons stand in for explanations.

Do instead: Select a consistent icon set and only include icons that improve recognition. Pair ambiguous icons with labels.

Acceptance check: Can the user understand the action without decoding a symbol?

08. Rainbow side-borders and colored stripes

Watch for: Every card has a differently colored leading rail even though the colors mean nothing.

Do instead: Map color to a documented category, status or severity; otherwise use neutral separators.

Acceptance check: Is there a legend or an obvious consistent meaning?

09. Meaningless status dots

Watch for: Pulsing green dots sit near headings, labels and navigation without an actual state.

Do instead: Only show states that are real, current, and comprehensible. Add a text label when necessary; remove decorative dots.

Acceptance check: What exactly changed when the dot changed color?

10. Endless centered stacks

Watch for: Heading, subtitle, illustration, button, cards and all copy are centered on every screen.

Do instead: Set readable edges, an intentional grid and asymmetric emphasis where useful. Center short hero copy selectively, not all information.

Acceptance check: Do the sections have purposeful alignment, not uniform centering?

11. Over-rounded everything

Watch for: Pill buttons, pill chips, pill cards and bubble sections blur component roles.

Do instead: Choose radii by component function and use a restrained geometric vocabulary.

Acceptance check: Are buttons, cards and indicators visually distinguishable?

12. Default component-library fingerprints

Watch for: Scaffolded shadcn-style layouts, unmodified components and the same card design on every route.

Do instead: Use libraries as engineering primitives; customize tokens, information structure, proportions, states and interaction behavior.

Acceptance check: Could this screen be mistaken for a starter template with its logo changed?

13. Arbitrary shadow and spacing

Watch for: Cards float at inconsistent elevations and every gap feels like a separate guess.

Do instead: Define spacing and elevation roles; use negative space rather than extra visual layers.

Acceptance check: Do sibling components obey one measurable spacing system?

14. Same visual composition for each section

Watch for: The landing page repeats hero, three-card grid, short text, three-card grid.

Do instead: Vary composition according to information: overview, live example, comparison, steps, supporting detail.

Acceptance check: Does the scroll reveal new information, not merely more identical boxes?

04 / CONTENT

Show, do not vaguely promise

A strong product interface explains what happens before, during and after a user action. Marketing copy, dashboards and onboarding should all be product-specific. [2, 3, 4]

Anti-patterns in words and evidence

15. Polished but meaningless copy

Watch for: "Unlock insights," "supercharge productivity," "seamless experience," "everything you need."

Do instead: Use exact nouns and verbs: import photos; review flagged rows; compare two sessions; export a report. Name inputs, outputs and constraints.

Acceptance check: Would the sentence still fit 500 unrelated products? If yes, rewrite.

16. Repeated marketing headings

Watch for: Every H2 starts with "Built for", "Everything you need", or "Designed to".

Do instead: Name an actual capability, objection or workflow. Read headings alone to test whether they tell a coherent product story.

Acceptance check: Can a visitor identify the product without reading body text?

17. Fake credibility signals

Watch for: Invented testimonials, customer logos, user counts, metrics or dashboards fill the page.

Do instead: Use genuine screenshots, test cases clearly labeled as samples, detailed walkthroughs, before/after examples, and limitations.

Acceptance check: Can every factual claim be verified or clearly labeled as illustrative?

18. Product never appears

Watch for: An ornate landing page states benefits without showing the interface or actual steps.

Do instead: Expose one truthful workflow: what users input, what runs, what they receive, and what they do next.

Acceptance check: Could a visitor explain how to use the tool after a brief skim?

19. Robot-like microcopy

Watch for: Controls say only "Submit"; errors say "Error"; empty states say "No data found."

Do instead: Use brief, concrete, human language: name the action, say what happened, provide the next safe step. Maintain one voice.

Acceptance check: Does every state help users continue, not merely report a status?

20. Stock AI imagery as product evidence

Watch for: Generic shiny 3D blobs, synthetic "team" pictures or invented interface shots masquerade as screenshots.

Do instead: Use actual interfaces and authentic imagery. If concept art is used, disclose or distinguish it and check for visual artifacts.

Acceptance check: Would a user incorrectly believe an illustration depicts existing functionality?

Rewrite examples

Generic default	Specific, verifiable alternative
"Transform your workflow"	"Add a check-in, attach photos, and compare two dates."
"AI-powered insights"	"Flags duplicate records and shows which rows need review."
"Get started"	"Create your first entry" / "Import a file" / "Compare photos"
"No data found"	"Nothing here yet. Add a photo to start your journal."
"Something went wrong"	"Could not save this entry. Your draft is still here; try again."

05 / LAYOUT

Make hierarchy visible

A screen needs an information story: first the task and outcome, then the controls and detail. Real design rhythm comes from changing density and arrangement according to meaning. [2, 4, 5]

For an application screen

- Lead with the user's current job, primary content or decision, not a decorative title.
- Keep primary controls close to the object they act on. Secondary actions should be quieter but still discoverable.
- When showing metrics, prioritize meaningful numbers and context; choose comparison types based on the data, not card symmetry.
- Use lists or tables for repeated comparable records; cards for independent objects; charts only when they reveal a pattern.
- Do not break platform interaction conventions merely to appear original.

For a landing or marketing page

- Hero: what this product does, for whom, and a clear next action. A real preview beats a glowing abstraction.
- First supporting section: a concrete workflow or product screenshot, not six generic benefit boxes.
- Middle: proof, mechanisms, edge cases, limitations and a reason to trust the product.
- Later: secondary capabilities, FAQs and technical detail. As users move down, information can become denser.
- Vary section structure where the content changes; repeat when repetition genuinely aids comprehension.

For a native or mobile app

- Respect navigation, gestures, touch targets, safe areas, system appearance and text scaling for that OS.
- Use short paths to common jobs; keep destructive actions clearly separated and confirmation proportionate.
- Prioritize real content on small screens. Avoid transplanting a desktop SaaS marketing grid into app navigation.
- Test portrait widths, one-handed reach where relevant, landscape/tablet behavior, and keyboard overlays.

Use variation carefully

Variety must be semantic, not random. A dashboard with four unrelated KPI card decorations is still a template; four deliberately different views of different kinds of information may be the right choice.

06 / STATES

Design the unglamorous screens

The clearest difference between a demo and a finished product is how it behaves when data, permission, connectivity, or user input is imperfect. These checks extend the article advice into an implementable QA standard. [3, 4, 5]

Every key flow needs these states

State	What an intentional design includes
First use / empty	Explain what belongs here, why it is empty and the next safe action. Avoid fabricated sample data masquerading as real.
Loading / progress	Explain what is happening. Use determinate progress if real; otherwise a restrained waiting state. No ornamental infinite pulses.
Error / recovery	Say what failed, what did not, and how to recover. Preserve unsaved work where possible.
Permission denied	State why access is useful, respect refusal and offer an alternative or settings route when possible.
Success / completion	Confirm the action and show its consequence; avoid gratuitous celebration for routine operations.
Long content / edge cases	Test clipped labels, extreme values, missing images, localization, zoom/text scaling and slow operations.

Accessibility and usability release expectations

□ Text and controls remain readable with adequate contrast; test both light and dark themes if supported.

□ Screen reader labels communicate the function of icons, inputs, errors and status.

□ Focus and keyboard navigation work on web/desktop; target sizes and gestures work on mobile.

□ Status uses text, shape or labels as well as color; success/error semantics do not rely on color alone.

□ Motion is subdued and honors reduced-motion preferences.

□ Responsive behavior is inspected at narrow, medium and wide sizes; no clipped or overlapping UI.

□ Every visible control has a working behavior or is clearly disabled with an explanation.

The sources emphasize intentionally designed zero/loading states and meaningful motion. Exact WCAG conformance and platform-specific measurements must be validated against the applicable standards; this PDF does not substitute for formal accessibility testing.

07 / METHOD

A workflow that produces judgment

Do not ask one model to invent the product strategy, interface, interactions, code and final QA in a single unreviewed step. Separate decisions into reviewable passes. [2, 4, 5]

Phase A — Research & reference

□ Collect two or three relevant references and document exactly which decisions are useful: reading density, navigation, list treatment, typography, empty state.

□ Do not copy a full competitor screen; combine patterns only when they solve the same user task.

□ Write the design brief and constraints. Distinguish facts from assumptions.

Phase B — Wireframe & direction

□ Outline information architecture, user flows and the primary action of each screen.

□ Draft at least two distinct layout approaches for the most important screen; critique both.

□ Select a direction and list the specific trade-offs it makes.

Phase C — Design system & implementation

□ Declare tokens for typography, color roles, spacing, corners, separators, interaction states and iconography.

□ Build the critical flow first, with real or clearly labeled sample content.

□ Use semantic components, shared tokens and maintainable responsive layout primitives. Avoid cosmetic one-off overrides that accumulate.

Phase D — Render, compare, refine

□ Inspect actual screenshots at phone, tablet and desktop sizes as relevant; do not rely on imagined results.

□ Run an anti-template pass: simplify nesting, eliminate gratuitous glow and badges, fix copy and typography, vary composition only with purpose.

□ Test unusual data, empty states, errors, keyboard/touch interactions and contrast.

□ Repeat until each major design issue has a concrete fix and a rechecked result.

Phase E — Add one earned signature detail

Introduce at most one or two small, product-related flourishes: a thoughtful transition tied to comparison, a distinctive timeline, a beautifully labeled control. It should improve recognition or delight without obscuring the task. Random "handmade" decoration is not a substitute for hierarchy. [4]

08 / REVIEW

The release gate

Use this as an evidence-based design review. Score each category, then fix the highest-risk failures before polishing secondary details. The rubric below is an editorial synthesis, not a published score from any source.

40-point UI quality gate

Product specificity (0–5)

Primary job obvious / Real workflow shown / Concrete copy / Truthful proof / Information reflects actual data

Hierarchy (0–5)

One focal point / Readable typography / Clear primary action / Effective spacing / Useful density

Visual discipline (0–5)

Limited palette / No unjustified gradients/glow / Cards earn their place / Badges/icons are meaningful / Consistent surfaces and radii

Layout (0–5)

Consistent alignments / No endless center stack / Appropriate list/table/card choice / Responsive layout / Purposeful rhythm

System consistency (0–5)

Reusable tokens / Consistent icon sizes / Uniform type roles / Controlled motion / Components match across routes

Content and honesty (0–5)

No fake metrics/testimonials / Clear buttons / Helpful microcopy / Specific headings / No placeholder marketing claims

Interaction states (0–5)

Thoughtful zero state / Loading feedback / Recovery from errors / Accessible focus/labels / Usable permission/success behavior

Verification (0–5)

Screens actually rendered / Mobile checked / Desktop/tablet checked / Edge cases checked / Major findings fixed and rechecked

How to score

Give 1 point for each observed, tested criterion above: maximum 40. **36–40:** strong candidate for release. **30–35:** focused refinement needed. **Below 30:** do another design pass. The score does not override critical usability failures.

Automatic blockers, regardless of score

- □ A primary action is broken, undiscoverable or misleading.
- □ Fake social proof or an invented product screenshot is presented as genuine.
- □ Text is unreadable, keyboard/touch use fails, or critical controls are inaccessible.
- □ A core state (empty, error, permission or loading) leaves users stuck.
- □ The AI claims to have visually inspected screens it did not render.

Evidence the AI should report

For every important finding: **screen / issue / why it harms the task / exact change / how to verify / verification status.** Report unresolved blockers and assumptions explicitly. Use a before/after screenshot when available.

09 / COPY AND PASTE

Master Build Prompt

Give this complete instruction to an AI app builder before implementation. Add your own product requirements in the marked fields. The rules are strict defaults, not inflexible prohibitions: purposeful exceptions require an explicit explanation.

Usage: Paste the entire prompt below into the AI's system/project instructions, an agent rules file, or the beginning of a new implementation conversation. Provide screenshots and platform details if available.

COPY-READY PROJECT INSTRUCTIONS

ROLE: You are the product designer, UX writer, accessibility-minded UI engineer, and visual QA reviewer for this application.

PRODUCT INPUTS (fill these before starting):

- Product and core jobs: [DESCRIBE ACTUAL USER TASKS].
- Audience, skill level and use context: [WHO / WHERE / WHY].
- Platform(s): [WEB / ANDROID / iOS / MAC / WINDOWS / OTHER].
- Key screens and critical flows: [LIST].
- Brand attributes and reference interfaces: [LIST OR SAY NONE].
- Technical, accessibility and content constraints: [LIST].

DESIGN MISSION:

Create an interface that is clearly designed for THIS product. Do not default to the statistical average of AI-made SaaS layouts. Clarity, consistency, real product evidence and task completion outrank decorative polish.

WORKING RULES:

1) Define the information architecture and primary user task for every screen before laying out components. Identify the one thing that should attract attention first.

2) Before coding, supply a short design direction: audience needs, personality, hierarchy, reference patterns, color roles, type roles, spacing, radii, icons and motion.

3) Build from shared semantic design tokens. Use intentional typography hierarchy; readable left-aligned long copy; consistent edges; predictable spacing and component behavior.

4) Use a neutral-first palette and at most one dominant brand accent by default. Status colors require meaning. Do not use rainbow accents, automatic purple/blue gradients, neon-on-dark styling or glow without a product reason.

5) Use dark mode only if appropriate and legible. No aurora backgrounds, glassmorphism, floating blur or shining button borders as automatic signs of "premium".

6) Avoid card-in-card nesting. A card must group independent, useful content. Prefer whitespace, dividers, rows, tables, lists and plain sections when they communicate more clearly.

7) Do not generate repetitive 3/6-card feature grids, identical icon tiles, generic dashboard KPIs or sections with the same composition simply to fill space. Vary form only when information varies.

8) Never use emoji as control/navigation icons. Use a consistent icon family sparingly, with accessible labels. Badges, colored side rails and status dots must communicate defined information.

9) Use truthful, product-specific text. Avoid empty language such as "unlock powerful insights," "seamlessly transform," "built for modern teams," and repetitive "Everything you need" headings. Name inputs, actions, outputs and limits.

10) Never fabricate customer logos, testimonials, usage statistics, dashboards or screenshots. If examples use invented data, label them clearly as samples. Prefer real workflow previews and honest proof.

11) Design first-use, empty, loading, error, permission, success and edge-case states. Write humane microcopy that tells users what happened and what to do next.

12) Respect platform conventions, responsiveness, touch targets, keyboard and screen reader usability, contrast, focus visibility, text scaling and reduced-motion preferences.

13) Avoid gratuitous infinite animation. Motion should describe transition, progress or feedback; not merely make a static layout look active.

14) Reuse stable components and tokens; do not force every content type into the same component. Do not leave starter-library aesthetics unchanged.

15) Add at most one or two product-specific visual touches after the structure works; explain their purpose. Do not invent decoration to appear human-made.

PROCESS:

A. Produce a brief with explicit assumptions and an inventory of screens/states.

B. Propose two coherent structures for the primary screen and recommend one with reasons.

C. Document the design system and implement the most important workflow end-to-end.

D. Render actual screens and review them at representative sizes. Do not claim rendering happened unless it did.

E. Audit for generic AI patterns, hierarchy, clarity, consistency, accessibility and truthful product evidence. Fix high-impact issues and re-inspect.

F. Report what was built, what was tested, unresolved risks and the rationale for non-obvious styling choices.

DELIVERABLE STANDARD:

Final UI must be recognizable as this product, coherent between screens and comprehensible in ordinary real-world states. If a visual element does not clarify, group, guide, identify or accurately signal state, challenge it or remove it.

10 / COPY AND PASTE

Redesign & Audit Prompt

Use this when a working app already looks generic. It tells the AI to find precise causes before changing the visual system, and to preserve real functionality.

Act as a senior product designer and UX engineer auditing an EXISTING application. Your objective is to remove generic AI-template design while preserving every real workflow.

1. Inspect the actual screens, source styling/components and current behaviors when accessible. If you cannot view or run them, say so and ask for screenshots or source context. Never pretend to have performed an audit.

2. Inventory issues under: product specificity; copy; information hierarchy; typography; color and contrast; card/wrapper overload; icon/badge/dot misuse; layout sameness; animation; credibility; empty/loading/error states; accessibility; responsive behavior.

3. Rank issues by impact: blocking usability, misleading trust claims, unclear task flow, visual hierarchy, cosmetic polish.

4. For every major issue produce: screen/location, evidence, why it reads as a default template, proposed precise replacement and how success will be checked.

5. Refactor to semantic tokens and reuse patterns. Remove unnecessary wrappers and decoration; replace vague copy with specific behavior; use actual content and workflow examples.

6. Produce a concise before/after difference list. Render at relevant device sizes. Report which checks were performed and which remain unverified.

> Never redesign solely by adding a new gradient, a trendy font, heavier shadows, more badges or more animation. Better design should be more meaningful, not busier.

A useful second-pass request

> Review the result again as if you did not build it. Name the five decisions that still look templated, explain why, and replace them with product-specific alternatives. Verify that those changes did not reduce clarity, accessibility or workflow efficiency. Be exact about any checks you could not run.

11 / SOURCES

Source map & editorial notes

This guide combines five user-provided PDFs. It is a **paraphrased synthesis and an original operating specification**, not a verbatim concatenation of the articles. Bracketed source numbers earlier in the guide identify influence, not line-by-line attribution.

[1] The Fountain Institute — Jeff Humble

7 Signs a UI Has Been Vibe Coded (And How to Avoid Them). Neon palettes, ornamental glow, emoji icons, purple gradients, over-cardification, rainbow side rails and meaningless status indicators.

https://www.thefountaininstitute.com/blog/signs-vibe-coded-ui

[2] r/vibecoding — community discussion

How do I make an app not look vibe coded?. Design thinking, research/benchmarking, plain-first iteration, real references, design skills, coherent themes, semantic frontend implementation and step-by-step review. Opinions in this thread are anecdotal and sometimes conflict.

https://www.reddit.com/r/vibecoding/comments/1sw9cjy/how_do_i_make_a_app_not_look_vibe_coded/

[3] VibeMole

How to Avoid Building Apps That Look Vibe Coded. 15 visual, language, structure and codebase signals; five focused improvements; product evidence and repeatable audit-and-fix process.

https://vibemole.com/resources/avoid-vibecoded-app-design

[4] Creative Mind Habits

How to Make Your Vibe-Coded App Not Look Vibe-Coded. Gradient restraint, real icons, alignment and typography, empty/loading states, genuine images, spacing rhythm, microcopy and a distinct crafted detail.

https://www.creativemindhabits.com/posts/2026-06-21-how-to-make-your-vibe-coded-app-not-look-vibe-coded/

[5] DEV Community — @kiwibreaksme

Your Vibe-Coded App Looks Ugly. Here's What I Did About It.. Written design rules, restrained colors and shadows, typographic ratios, layout rhythm, design tokens/skins and a staged UX review. The article gives examples from a larger rule set, not all 69 rules.

https://dev.to/kiwibreaksme/your-vibe-coded-app-looks-ugly-heres-what-i-did-about-it-2nb4

Scope and important qualifications

- The five sources concern mainly web apps and SaaS landing pages. Native-app conventions, accessibility checks, numeric tokens, scoring, and the complete AI prompts are implementation-oriented additions in this playbook.
- Examples such as a 4/8-point spacing scale or 6/10/16 radii are **proposed baselines**, not established requirements and not claims made by the authors.
- A purple gradient, Lucide icon, Tailwind class, card, dark theme, status dot or centered section is **not inherently bad**. It becomes generic when overused without purpose.
- A clean look alone is not enough. A product can appear unique and still be difficult to use; task success, honest evidence and accessibility come first.

Final instruction to the AI: Build the interface that this workflow deserves, not the interface most likely to appear after the prompt "make a modern app."